

Materia

Seguridad en el desarrollo de Aplicaciones

Nombre

Ricardo Porras Reyes

Matricula

2024171008

Grupo

IDSG17

Profesor

Gerardo Ramírez Villarreal

Objetivo

Utilizar Grafana y el lenguaje de consultas LogQL para consultar, filtrar y analizar los registros generados por VulnerableApp almacenados en Loki. Además, construir un dashboard que facilite el monitoreo del sistema y el análisis de incidentes relacionados con autenticación, auditoría y errores.

Desarrollo de la práctica

Verificación de la plataforma

Primero verifiqué que los contenedores de Grafana, Loki y Promtail continuaran en ejecución mediante el comando `docker ps`. Después inicié VulnerableApp y confirmé que los registros seguían enviándose correctamente tanto a Seq como a Loki sin modificar la configuración implementada en las prácticas anteriores.

Evidencia

- Captura del comando `docker ps`.
- Captura de Grafana funcionando.
- Captura de Seq mostrando eventos.
- Captura de VulnerableApp en ejecución.

Exploración de Grafana

Ingresé al apartado **Explore** de Grafana y seleccioné Loki como fuente de datos para realizar consultas sobre los registros.

Componentes identificados

Elemento	Descripción
Editor de consultas	Permite escribir consultas utilizando LogQL.
Selector de tiempo	Define el intervalo de tiempo sobre el que se realizará la búsqueda.
Panel de resultados	Muestra todos los registros encontrados que cumplen con la consulta realizada.

Sintaxis básica de LogQL

Consulta	Propósito	Resultado obtenido
<code>{application="VulnerableApp"}</code>	Mostrar todos los registros generados por la aplicación.	Se visualizaron todos los eventos almacenados en Loki.
<code>{application="Security"}</code>	Consultar eventos relacionados con seguridad.	Se mostraron eventos del módulo de seguridad.
<code>{application="Audit"}</code>	Consultar registros de auditoría.	Se visualizaron las acciones registradas por el sistema.
<code>`{application="VulnerableApp"}</code>	<code>= "Error"</code>	Filtrar únicamente los errores.
<code>`{application="VulnerableApp"}</code>	<code>= "Warning"</code>	Filtrar advertencias.
<code>`{application="VulnerableApp", module="Authentication"}</code>	<code>= "Login"</code>	Combinar labels con búsqueda de texto.

Consultas específicas

Para comprobar el funcionamiento de LogQL realicé diferentes consultas orientadas al monitoreo del sistema.

Errores

```
{application="VulnerableApp"} |= "Error"
```

Esta consulta permite visualizar únicamente los eventos registrados como errores.

Evidencia: Captura de la consulta.

Advertencias

```
{application="VulnerableApp"} |= "Warning"
```

Con esta consulta identifiqué los eventos clasificados como advertencias.

Evidencia: Captura correspondiente.

Eventos de autenticación

```
{module="Authentication"}
```

Esta consulta recupera todos los registros generados durante el proceso de autenticación.

Evidencia: Captura correspondiente.

Excepciones

```
{application="VulnerableApp"} != "Exception"
```

Permite localizar rápidamente las excepciones generadas por la aplicación.

Generación de actividad

Para generar evidencia suficiente realicé diversas operaciones dentro de VulnerableApp:

- Inicio de sesión exitoso.
- Inicio de sesión con credenciales incorrectas.
- Altas de registros.
- Consultas.
- Actualizaciones.
- Eliminaciones.
- Generación de errores mediante entradas inválidas.

Estas acciones produjeron suficientes registros para realizar el análisis posterior.

Análisis del incidente

Se reportó un incremento de errores de autenticación, por lo que utilicé LogQL para investigar el problema.

¿Cuándo comenzaron?

Mediante la consulta:

```
{application="VulnerableApp"} != "Login failed"
```

identifiqué el primer evento registrado de autenticación fallida y la hora exacta en la que ocurrió.

¿Qué módulo los produjo?

La consulta:

```
{module="Authentication"}
```

mostró que todos los eventos provenían del módulo de autenticación.

¿Qué usuarios estuvieron involucrados?

Busqué los nombres de usuario registrados dentro de los mensajes y confirmé qué cuentas realizaron intentos de acceso.

¿Qué excepción se registró?

Con la consulta:

```
{application="VulnerableApp"} |= "Exception"
```

identifiqué las excepciones relacionadas con el proceso de autenticación.

Evidencia

Las consultas permitieron observar:

- Hora del incidente.
- Usuario involucrado.
- Módulo que generó el evento.
- Mensaje registrado.
- Nivel de severidad.

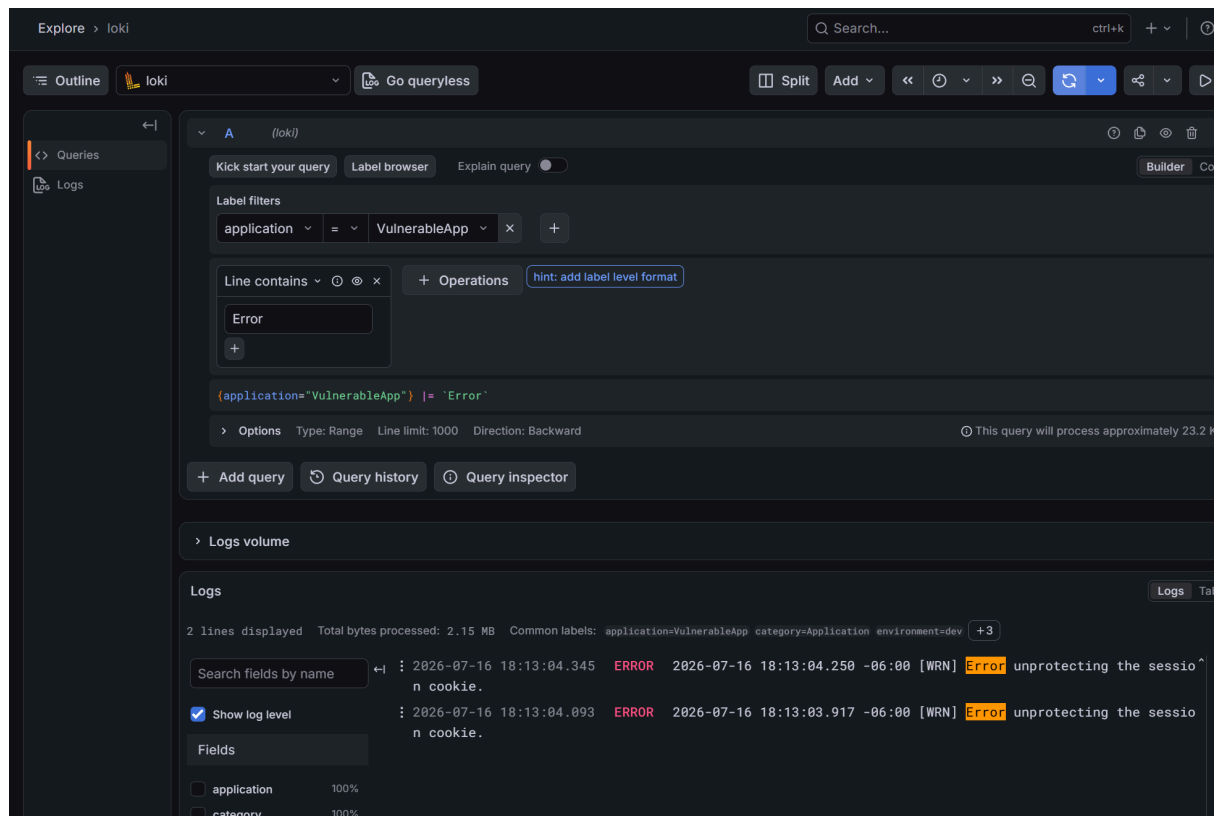
Construcción del dashboard

Se creó un dashboard de monitoreo integrado por tres paneles.

Panel 1. Errores

Consulta utilizada:

```
{application="VulnerableApp"} |= "Error"
```



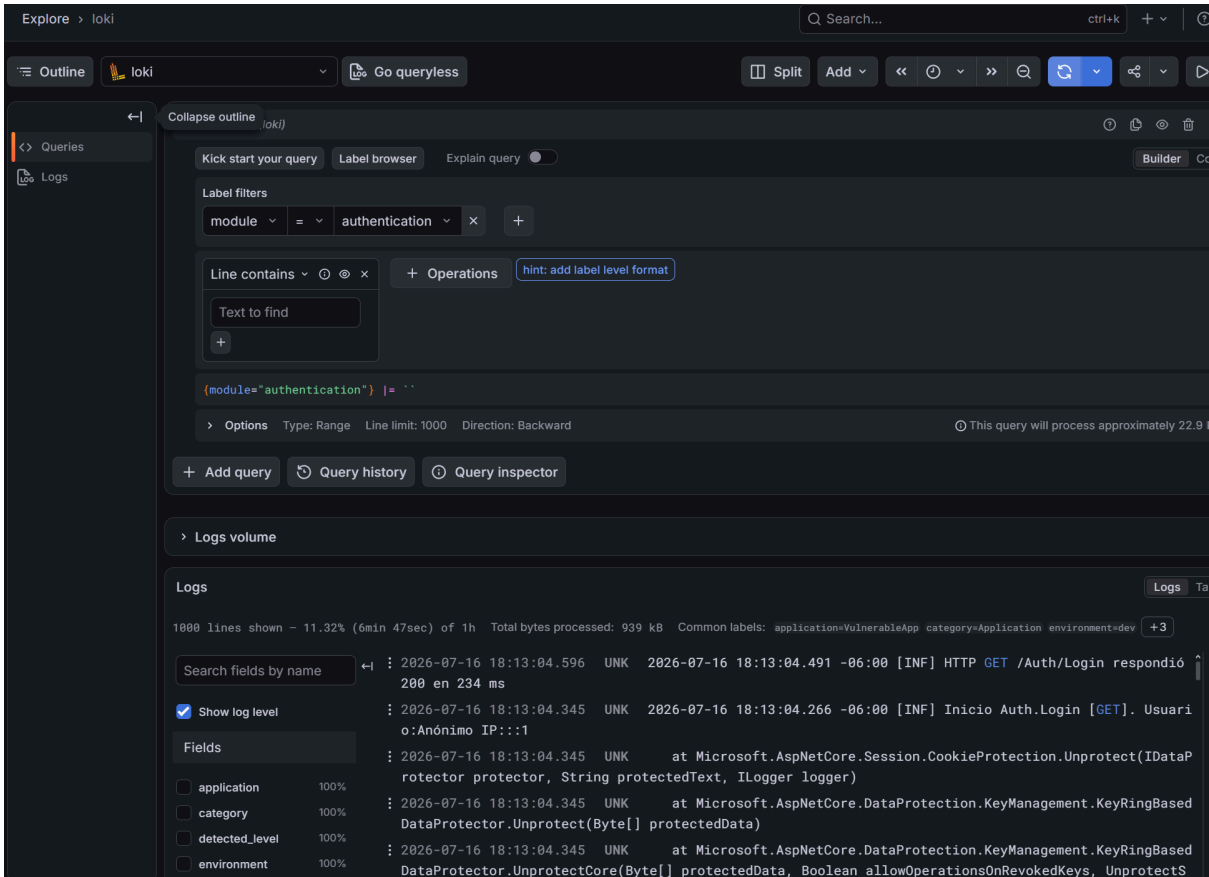
Propósito

Visualizar rápidamente los errores registrados por la aplicación para detectar posibles fallas.

Panel 2. Autenticaciones

Consulta utilizada:

`{module="Authentication"}`



Propósito

Monitorear todos los eventos relacionados con el inicio de sesión de los usuarios.

Comparación entre Seq y Grafana

Característica	Seq	Grafana + Loki
Búsqueda textual	Sí	Sí
Filtros por labels	Limitados	Sí, mediante LogQL
Dashboards	Básicos	Avanzados y personalizables
Investigación de incidentes	Buena	Muy completa

Uso principal	Visualización y consulta de eventos	Monitoreo y observabilidad
---------------	-------------------------------------	----------------------------

Caso de estudio

¿Cuándo comenzaron los errores?

Los errores comenzaron a registrarse desde el primer intento de autenticación fallido observado en Grafana dentro del intervalo de tiempo analizado.

¿Qué módulo los generó?

Los registros muestran que el módulo **Authentication** fue el responsable de generar dichos eventos.

¿Qué usuarios aparecen involucrados?

Los mensajes registrados permitieron identificar a los usuarios que realizaron los intentos fallidos de autenticación.

¿Qué excepción se registró?

Se registró una excepción relacionada con credenciales inválidas durante el proceso de autenticación.

Evidencia

Las consultas realizadas en Grafana muestran el horario del evento, el usuario involucrado, el módulo responsable y el mensaje registrado, información suficiente para reconstruir el incidente.

Pregunta de reflexión

LogQL ofrece importantes ventajas frente a revisar directamente archivos de texto o realizar búsquedas simples en Seq. Gracias al uso de labels es posible filtrar información de manera más precisa, reutilizar consultas y construir dashboards que muestran información en tiempo real. Esto facilita detectar incidentes de seguridad, identificar tendencias y reducir el tiempo necesario para investigar problemas dentro de una aplicación.

Reto

Se diseñó un dashboard de monitoreo de seguridad compuesto por tres paneles:

- Autenticaciones fallidas.
- Eventos de auditoría.
- Errores críticos.

Este dashboard permite al personal de soporte identificar rápidamente intentos de acceso no autorizados, revisar las acciones realizadas por los usuarios y detectar errores que puedan afectar la disponibilidad del sistema. Su utilización facilita el monitoreo continuo y acelera la respuesta ante incidentes de seguridad.

The screenshot shows the Grafana dashboard interface. The left sidebar contains navigation links: Home, Bookmarks, Starred, Dashboards (selected), Explore, Drilldown, Metrics, Logs, Traces, Profiles, Alerting, Connections, and Administration. The main panel displays log data for the 'Env=Dev' dashboard. The logs are organized into three sections: 'Env=Dev', 'Module=Auth', and 'Posts'. The 'Env=Dev' section shows log entries for 'Auth/Login' and 'Auth/Login' with log levels 'INF' and 'WRN'. The 'Module=Auth' section shows log entries for 'Auth/Login' and 'Auth/Login' with log levels 'INF' and 'WRN'. The 'Posts' section shows log entries for 'Auth/Login' and 'Auth/Login' with log levels 'INF' and 'WRN'. The 'Errors' section highlights critical errors related to unprotecting the session.

Time	Level	Message
2026-07-16 18:13:04.491	INF	HTTP GET /Auth/Login res
2026-07-16 18:13:04.266	INF	Inicio Auth.Login [GET].
2026-07-16 18:13:04.266	WRN	at Microsoft.AspNetCore.Session.CookieProtection.Unprotect
2026-07-16 18:13:04.266	WRN	at Microsoft.AspNetCore.DataProtection.KeyManagement.KeyRi
2026-07-16 18:13:04.266	WRN	at Microsoft.AspNetCore.DataProtection.KeyManagement.KeyRi
2026-07-16 18:13:04.266	WRN	at Microsoft.AspNetCore.DataProtection.Cng.Internal.CngAut
2026-07-16 18:13:04.266	WRN	at Microsoft.AspNetCore.DataProtection.Cng.CbcAuthenticate
2026-07-16 18:13:04.266	WRN	System.Security.Cryptography.CryptographicException: The payl
2026-07-16 18:13:04.250	WRN	Error unprotecting the s
2026-07-16 18:13:03.917	WRN	Error unprotecting the s